

What is Container

- **A container is a lightweight, portable, and isolated environment for running applications.**
- **It packages an application and its dependencies (libraries, runtime, configuration files, etc.) together, ensuring that the application runs consistently across different computing environments.**
- **Containers share the host operating system's kernel, making them more efficient than traditional virtual machines (VMs) in terms of resource usage.**

Without Containers

- You'd manually set up the servers (physical or virtual) to host your website
- Setting up an operating system and configuring the necessary dependencies (e.g., libraries, frameworks).
- Manually installing updates and patches to keep the system secure and compatible.
- You need to install and configure the web server (e.g., Nginx, Apache) and application server.
- This involves setting up environment variables, installing language runtimes (e.g., Node.js, Python), and other dependencies.
- Scaling traditionally requires provisioning additional servers or VMs, configuring them with the same environment, and setting up load balancing.
- Continuous Integration and Continuous Deployment (CI/CD) may be more complex without standardized environments.

How containers improve their cloud-based infrastructure

- **Application Packaging:** It packages its frontend, backend, and database services into separate containers.
- **Microservices Architecture:** It organizes the website into different microservices, such as Product Catalog, User Authentication, Payment Processing, and Order Management.
- **Scalability:** During peak times, the Product Catalog and Payment Processing containers can be automatically scaled up by creating more instances to handle the load.
- **Cost Efficiency:** Since containers use shared OS resources, they are more lightweight and consume fewer resources than VMs.

Feature	Virtual Machines (VMs)	Containers
Operating System	Each VM includes a full OS	Shares host OS kernel
Size	Larger (GBs), as it includes OS	Smaller (MBs), as it only includes app and dependencies
Boot Time	Slower (minutes)	Faster (seconds or milliseconds)
Isolation	Full isolation with separate OS	Process-level isolation, shares OS kernel
Resource Utilization	More resource-intensive	Less resource-intensive, higher density
Ideal Use Cases	Applications needing full OS or legacy apps	Microservices, cloud-native applications, DevOps
Examples	VMware, Microsoft Hyper-V, KVM	Docker, Kubernetes, Podman